

AI Job Search Copilot — Design Document

Multi-Agent System Architecture — March 27, 2026

What This Project Does

The idea behind this project is pretty simple: job hunting involves a lot of repetitive email checking, and it's easy to lose track of where you are with each company. So we built a system that connects to your Gmail, pulls out the job-related emails, figures out what stage each one is at (like whether it's an interview invite or a rejection), and keeps a running tracker of all your applications. There's also a resume tool that compares your resume to a specific job posting and tells you what to fix.

Under the hood, the app is split into five “agents” that each handle one piece of the process. Three of them work together in a chain to handle emails, and the other two deal with resumes. We used Python with FastAPI for the backend, SQLite for the database, and the OpenAI Agents SDK to wire up the LLM-powered parts. Users log in through Google OAuth, which also gives us permission to read their Gmail.

The Five Agents

Here's a quick summary of what each agent does, what it takes in, and what it spits out:

#	Agent	Type	Input	Output
1	Gmail Intake	Deterministic	User's OAuth token	List of emails (sender, subject, date, body)
2	Email Classifier	LLM (gpt-4o-mini)	Emails from Agent 1	Classification for each email (stage, company, role, confidence)
3	Tracker Updater	Deterministic	Classifications from Agent 2	Counts of apps created, updated, or skipped
4	Resume Reviser	LLM (gpt-4o)	Resume text + job description	Keyword matches, gaps, and bullet rewrites
5	Critic	LLM (gpt-4o)	Another agent's output + source data	Verdict (approved, flagged, or needs correction)

Agent 1 (Gmail Intake) is the starting point. It hits the Gmail API with a targeted search query that only looks at the Primary inbox tab for emails from the last 24 hours. It filters by job-related keywords in the subject line (things like “interview,” “application,” “offer”) and by known recruiting platform domains like greenhouse.io and lever.co. It skips any emails we've already seen, strips out the HTML formatting, and saves the clean version to our database.

Agent 2 (Email Classifier) is where the LLM comes in. It takes the emails from Agent 1 and sends them to GPT-4o-mini in small batches of 5 at a time. For each email, it figures out the company name, the role, and what hiring stage it represents — things like Applied, Assessment, Interview, Offer, or Rejection. We went with gpt-4o-mini instead of the full model because we kept hitting rate limits when sending all 50 emails at once to the bigger model. There's a 2-second pause between batches to stay under the token limit.

Agent 3 (Tracker Updater) doesn't use an LLM at all — it's just logic. It takes the classifications and decides what to do with each one. If we haven't seen that company/role combo before, it creates a new entry in the tracker. If we have, it only updates the stage if the new one is further along in the pipeline (so it won't accidentally move you backwards from “Interview” to “Applied”). It also sets a suggested next action for each stage, like “prepare for interview” or “review offer details.”

Agent 4 (Resume Reviser) handles the resume side of things. You paste in your resume and a job description, and it compares the two. It pulls out which keywords you already have, which ones

you're missing, and then suggests specific rewrites for your bullet points. The important thing is it only reframes what you've actually done — it won't make up experience you don't have.

Agent 5 (Critic) acts as a quality checker for the other agents. It can review a classification to see if the company name was hallucinated or if the stage label doesn't match what the email actually says. It can also review a resume revision to catch generic advice or fabricated experience. It's basically our sanity check layer.

How the Agents Talk to Each Other

The agents don't communicate over the network or anything like that. They're just Python functions that get called one after another inside a pipeline file. The output of one function gets passed directly as the input to the next one. There are two pipelines:

```
Email Pipeline (runs Agents 1 -> 2 -> 3):
  1. run_gmail_intake(db, user) -> list of email dicts
  2. run_email_classifier(emails, db) -> list of classification dicts
  3. run_tracker_updater(classes, db, id) -> {created, updated, skipped}

Resume Pipeline (runs Agent 4, optionally Agent 5):
  1. run_resume_reviser(text, jd) -> revision result
  2. review_resume_revision(...) -> critic verdict (optional)
```

The email pipeline gets kicked off either when the user clicks “Scan Inbox” in the UI, or automatically every 24 hours by a background scheduler that loops through all users who have connected their Gmail. The resume pipeline only runs when someone submits a resume through the frontend.

The database is the shared state between all the agents. Agent 1 writes emails to it, Agent 2 updates those emails with classification results, and Agent 3 creates or updates application records based on those results.

How We Handle Failures

Things go wrong, so we tried to make sure the system doesn't just crash when they do. Here's a breakdown:

Agent	What Can Go Wrong	What We Do About It
Gmail Intake	OAuth token expires	We try to auto-refresh it. If that fails, we tell the user to reconnect Gmail.
	No emails found	Pipeline just stops early — no point running the classifier on nothing.
	Messy HTML or bad dates	We fall back to raw text or use the current timestamp. The email still gets saved.
Classifier	OpenAI rate limit (429)	Small batches of 5 with 2-second pauses between them keeps us under the limit most of the time.
	A batch fails entirely	We mark those emails with confidence 0 and a “retry” flag. The rest of the batches keep going.
	Low confidence scores	We still pass them through, but the frontend shows a warning if overall confidence drops below 70%.
Tracker	Can't tell if it's a duplicate	We do case-insensitive matching on company + role. Not perfect, but works for most cases.
	Tries to go backwards in stage	Hard no — we just skip the update and add a note instead.
Resume Reviser	LLM call fails	The error gets passed back to the frontend as-is so the user knows what happened.
	Makes stuff up	The prompt tells it not to, and the Critic agent is there to catch it if it does.
Critic	Critic itself fails	It's optional, so the main results still go through. It just degrades gracefully.

On the security side, every API endpoint requires a JWT token, so you can't access someone else's data. Gmail tokens are encrypted before we store them in the database using Fernet encryption. And the scheduler wraps each user's scan in a try/except, so if one user's Gmail connection is broken it doesn't stop everyone else's scans from running.

Database Tables

We have four main tables, and they're all tied to a specific user through a foreign key so everyone's data stays separate:

Table	Key Fields	What It Stores
<code>users</code>	<code>email</code> , <code>hashed_password</code> , <code>gmail_token_encrypted</code>	User accounts and their encrypted Gmail tokens
<code>emails</code>	<code>user_id</code> , <code>message_id</code> , <code>classified_stage</code>	The raw emails we pulled plus their classifications
<code>applications</code>	<code>user_id</code> , <code>company</code> , <code>role</code> , <code>current_stage</code> , <code>status_history</code>	The actual application tracker entries
<code>resume_sessions</code>	<code>user_id</code> , <code>resume_text</code> , <code>job_description</code> , <code>revision_output</code>	History of resume analyses